

Submit your answers by adding and committing one file with your answers into the `hw2` directory of your CNETID-cs144-aut-24 svn repository. The file should be named either `hw2.txt` or `hw2_sol.pdf` for answers written in a plain ASCII text file or PDF file, respectively. PDFs of scanned hand-written pages must not exceed 5 megabytes. No other file formats or filenames are acceptable, and no files besides `hw2.txt` and `hw2_sol.pdf` will be graded. Not following directions may result in losing points.

Note: Some of the operands and instructions below use the scaled indexed addressing mode. As the caption for Figure 3.3 in the book explains, for x86 the valid scaling factors are 1, 2, 4, or 8. For this homework and this homework only, however, you should assume that there are no restrictions on scaling factors.

**(Q1) [20 points]** Assume the same information about addresses, registers, and values as in Q2 of homework 1. The table is reprinted here for convenience:

| Address | Value | Register | Value |
|---------|-------|----------|-------|
| 0x210   | 0xAB  | %rax     | 0x210 |
| 0x218   | 0xBC  | %rcx     | 0x220 |
| 0x220   | 0x09  | %rdx     | 0x1   |
| 0x228   | 0x42  | %rsi     | 0x8   |
| 0x230   | 0x54  | %rdi     | 0x100 |

Each part of this question concerns a single instruction. Each instruction is to be considered in isolation (the answer for one instruction does not affect any other instruction). The argument ordering for the instructions is the same ATT format (not Intel format) as used in the book. Negative values are represented with two's complement.

For each part, give the Destination for the instruction (as either “0xADDR” for address ADDR or as “%reg” for register reg), and the (unsigned) Value (**in hex**, starting with “0x”) stored in the Destination. If the facts given don't provide sufficient information to answer the question unambiguously, give “unknown” for Destination, and a terse explanation why for Value.

| Part | Instruction                              | Destination | Value |
|------|------------------------------------------|-------------|-------|
| e.g. | <code>addq %rdx, (%rax)</code>           | 0x210       | 0xAC  |
| A.   | <code>incq %rsi</code>                   |             |       |
| B.   | <code>xorq %rbx, %rbx</code>             |             |       |
| C.   | <code>decq (%rax)</code>                 |             |       |
| D.   | <code>subq 0x228, %rdi</code>            |             |       |
| E.   | <code>notq %rdx</code>                   |             |       |
| F.   | <code>shlq %rsi, (%rcx)</code>           |             |       |
| G.   | <code>subq \$0x100, %rax</code>          |             |       |
| H.   | <code>imulq %rdi, (%rax, %rsi)</code>    |             |       |
| I.   | <code>orq (%rcx), %rax</code>            |             |       |
| J.   | <code>shrq %rdx, (%rdi, %rsi, 38)</code> |             |       |

**(Q2)** (10 points)

Consider the following assembly code:

```
1      .globl  _loop
2  _loop:
3      xorq    %rax, %rax
4      movq    $4, %rdx
5  foo:
6      movq    %rax, %rcx
7      movq    %rdx, %rax
8      andq    %rdi, %rax
9      orq     %rcx, %rax
10     shlq    %rsi, %rdx
11     testq   %rdx, %rdx
12     jne     foo
13     ret
```

The assembly code was generated by compiling C code with the following overall form:

```
long loop(long x, long n) {
    long result = __1__;
    long mask;
    for (mask = __2__; mask __3__; mask = __4__) {
        result __5__;
    }
    return result;
}
```

Your task is to fill in the missing parts of the C code to get a program equivalent to the generated assembly code. Recall that the result of the function is returned in register `%rax`. You will find it helpful to examine the assembly code before, during, and after the loop to form a consistent mapping between the registers and the program variables. The clarity of your answers below may be improved by mentioning assembly line numbers.

- A. Which registers hold program values `x`, `n`, `result`, and `mask`?
- B. What are the initial values of `result` and `mask`?
- C. What is the test condition for `mask`?
- D. How does `mask` get updated?
- E. How does `result` get updated?
- F. Fill in all the missing parts of the C code, by providing the entire contents of the `__1__`, `__2__`, etc blanks.

**(Q3)** (10 points)

Your task is to **Manually** decompile the following assembly code into two short C functions `funcQ()` and `funcP()`. You decompile the assembly by using **your brain**, powered by your understanding of assembly (from lectures and Chapter 3). There is no single correct representation of a C function in assembly.

```
1      .globl _funcQ
2  _funcQ: # long funcQ(long x, long y)
3          # %rdi = x
4          # %rsi = y
5          imulq $3, %rsi, %rax
6          addq %rdi, %rax
7          ret
8      .globl _funcP
9  _funcP: # long funcP(long r, long s, long t)
10         # %rdi = r
11         # %rsi = s
12         # %rdx = t
13         testq %rsi, %rsi
14         jle   foo
15         movq  %rdx, %rax
16         movq  %rdi, %rdx
17         movq  %rax, %rdi
18         callq _funcQ
19         addq  %rdx, %rax
20         jmp   bar
21 foo:
22         movq  %rdi, %rax
23         movq  %rsi, %rdi
24         movq  %rax, %rsi
25         callq _funcQ
26         addq  %rdx, %rax
27 bar:
28         ret
```

Note: For this homework, it is fine to use a calculator or some other program to aid with arithmetic.

**REMINDER - STARTING IN 2022, WE HAVE STRICTER POLICIES ON PROJECT AND HOMEWORK SUBMISSIONS:**

Please triple check that you have submitted the right file(s), the right filename, the right content, the right format, and the right version.

We recommend submitting 15 minutes before the deadline (or even earlier) so you have 15 minutes to double/triple check your submission.

We will not accommodate incorrect or late submissions. 11:59:59pm is the hard deadline.